



Lecture[1]

C FUNDAMENTALS



Agenda

- Data type and storage
- Expression
- Control flow



Agenda

- Data type and storage
- Expression
- Control flow



Data Type

- C has the following simple data types:

Data type	C code	Size in bytes	Range
Char or signed char	char	1	-128 to 127
Unsigned char	unsigned char	1	0 to 255
int or signed int	int	at least 2	-32768 to 32767
Unsigned int	unsigned int	at least 2	0 to 65535
Short int or signed short int	short	2	-32768 to 32767
Unsigned short int	unsigned short	2	0 to 65535
Long int or Signed long int	long	at least 4	-2147483648 to 2147483647
Unsigned long int	unsigned long	at least 4	0 to 4294967295
float	float	4	3.4E-38 to 3.4E+38
double	double	8	1.7E-308 to 1.7E+308
Long double	long double	10*	3.4E-4932 to 1.1E+4932

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6
7      int b=3;
8      //print out the value of b
9      printf("the size of b is %u", sizeof(b));
10     return 0;
11 }
```

sizeof 是一个编译时运算符而非函数，它的核心作用是在编译阶段就确定其操作数（可以是数据类型或具体变量）所占用的内存字节数。

the size of b is 4

On 32-bit machine is usually 4 bytes (32bits)



- **#include**是一个预处理指令，其核心作用是在编译代码之前，将指定文件（通常是头文件）的全部内容原封不动地“复制粘贴”到当前指令所在的位置；这使我们能够使用标准库函数（如 printf 需要通过 #include <stdio.h> 引入其声明）或共享自定义的代码模块（使用 #include "myheader.h"），从而极大地提高了代码的复用性和项目结构的清晰度。
- **stdio.h**是标准输入输出（Standard Input & Output）的头文件，它提供了与外部设备进行数据交换的核心函数，使得程序能够从键盘读取输入（如使用 scanf）、向屏幕输出结果（如使用 printf）、以及对文件进行读写操作（如使用 fopen 和 fclose），是实现用户交互和数据持久化最基本且必不可少的工具集。
- **stdlib.h**是标准库（Standard Library）的头文件，它提供了大量通用且至关重要的函数，包括内存管理（如 malloc 和 free 用于动态分配和释放内存）、随机数生成（rand）、字符串与数字的转换（atoi）、进程控制（如 exit 用于终止程序）、以及环境交互（如 system 执行系统命令）等功能，是进行除基础输入输出外各种系统级和通用编程操作的核心工具集。



Unsigned int

- How to store an unsigned int
 - All bits are used to store value

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$$1111_2 = 15_{10}$$

- 2 Bytes = 16 bits
- $0 \sim 2^{16} - 1 = 65535$



int with sign

- Need to both represent positive and negative int
- Sign and magnitude (原码)
 - Use highest bit for sign: 0 -> +, 1 -> -
 - Other bits are used to store value

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$$1111_2 = 15_{10}$$

– But

- What about 0? Two representations
- $15 + (-15) = ?$ $00001111 + 10001111 = ?$

1	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$$1000111_2 = -15_{10}$$



One's-Complement for signed numbers

- One's-Complement (二进制反码)

- Use highest bit for sign: 0 -> +, 1 -> -
- Positive number X is encode of X
- Negative number X is encode $\sim X$ and highest bit is 1

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

15_{10}

1	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---

-15_{10}

– But

- What about 0? Two representations
- $15 + (-15) = ?$ $00001111 + 11110000 = ?$



Two's-Complement for signed numbers

- Two's-Complement (二进制补码)
 - Use highest bit for sign: 0 -> +, 1 -> -
 - Positive number X is encode of X
 - Negative number X is encode of $2^n + X$ and highest bit is 1

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

15_{10}

1	1	1	1	0	0	0	1
---	---	---	---	---	---	---	---

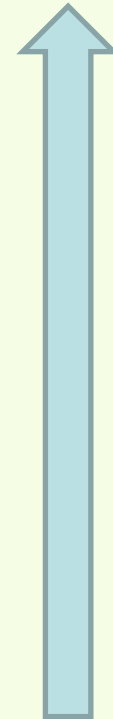
-15_{10}

(方法2: 反码+1)



Two's-Complement for signed numbers

- Only one 0
- $-2^{n-1} \sim 2^{n-1}-1$
- The order is respected
- Arithmetic calculation



Two's complement	Decimal
[0]111	+7
[0]110	+6
.....	
[0]001	+1
[0]000	0
[1]111	-1
.....	
[1]001	-7
[1]000	-8



- 原码、反码和补码是计算机用来表示有符号整数的三种编码方式;
- **原码**最直观，最高位表示符号（0正1负），其余位表示绝对值，但会导致“零”有两种表示形式 (+0 和 -0) 且加减运算复杂;
- **反码**是在原码基础上对负数各位取反得来，虽然简化了运算但“零”的问题依然存在;
- **补码**则完美解决了这些问题，它是在反码基础上加1得到，使得加减运算可以统一用加法器完成，“零”只有唯一表示，并且能多表示一个最小值，因此成为了现代计算机中有符号整数存储的实际标准方案。
- 对于正数： 原码 = 补码 = 反码（三者相同）
- 对于负数： 原码 $\rightarrow$ 补码：符号位不变，数值位取反+1
补码 $\rightarrow$ 原码：符号位不变，数值位取反+1



Floating types

float

single-precision floating-point

double

double-precision floating-point

long double

extended-precision floating-point

/	Smallest Positive Value	Largest Value	Precision	Size
float	1.17×10^{-38}	3.40×10^{38}	7 digits	4 Bytes
double	2.22×10^{-308}	1.79×10^{308}	15 digits	8 Bytes

float x;
scanf("%f", &x);
printf("%f", x);

double x;
scanf("%lf", &x);
printf("%f", x);

long double x;
scanf("%Lf", &x);
printf("%Lf", x);



How to represent a float

- How to represent a decimal fraction in binary?

– $10.125 \rightarrow 1010.001_2$

- Formatted form

– $1010.001_2 = 0.1010001_2 \times 2^{100}_2$

- Encoding of float

– $N = (-1)^S \times 0.M \times 2^E$

$$\begin{array}{l} 0.125 * 2 = 0.25 \\ 0.25 * 2 = 0.5 \\ 0.5 * 2 = 1.0 \end{array}$$

一共将小数点**向左**
移动了4位，4的二
进制是100

S	E	M
0	000100	001010001

10.125 in 16 bits



IEEE 754 float (4 Bytes)

符号位 (S)

指数位 (E)

尾数位 (M)



1 bit

8 bits

23bits

- M is consider as 0.1xxxx->1.xxxx
– Save one bit

- E use $E = E_{\text{real}} + 127$

- $N = (-1)^S \times (1 + M) \times 2^{E-127}$

M是23位尾数位表示的二进制小数。这里有一个隐藏的**前导1**，所以实际的值是1.xxx。因为这个前导的1总是存在，所以为了节省一位宝贵的存储空间，在将数字存入内存时，这个1被省略（隐含）了。在从内存中读取数字进行计算时，这个1会被自动加回去。

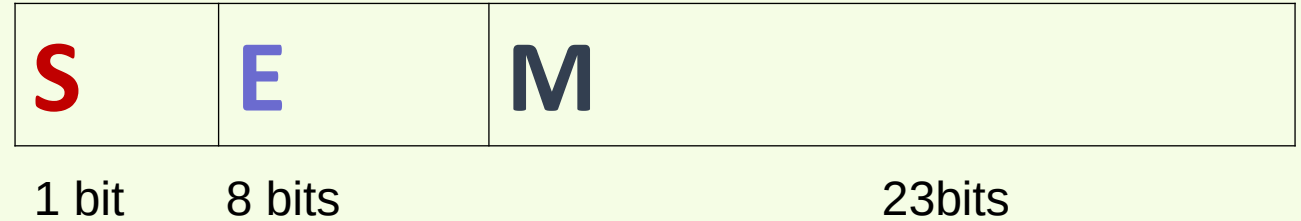
这里的 127 称为**指数偏移 (Exponent Bias)**。8位指数本身是无符号数（范围 0-255），减去127后，实际指数范围是 **-126 到 +127**。



IEEE 754 float (4 Bytes)

- $N = (-1)^S \times (1+M) \times 2^{E-127}$

- The range is



- Smallest positive value: (规约数, IEEE 754标准)

- 符号位 $S = 0$ (正数)

- 指数位 E 尽可能小: E 不能全为0 (全为0表示次正规数), 所以最小为00000001 (1)。

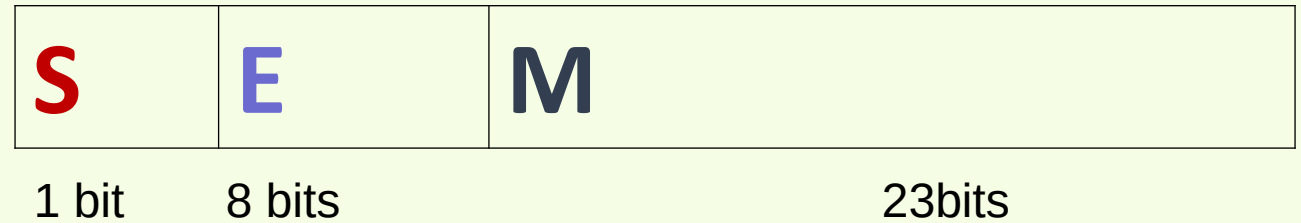
- 尾数位 M 尽可能小: 23位全部为 0, $M=1.00...0$

- $N_{\min} = 1.0 \times 2^{1-127} = 2^{-126} \approx 1.17549435 \times 10^{-38}$



IEEE 754 float (4 Bytes)

- $N = (-1)^S \times (1+M) \times 2^{E-127}$
- The range is

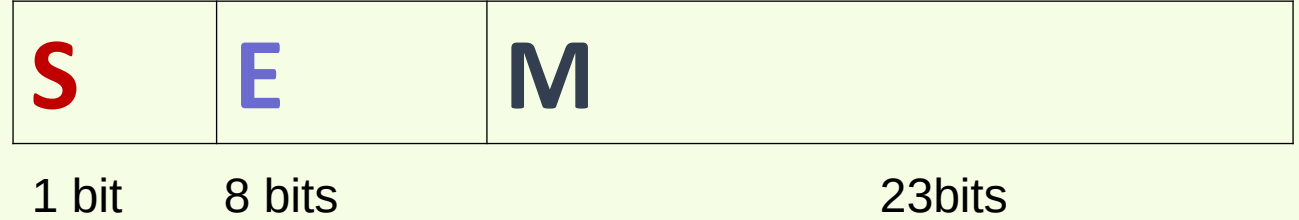


– Biggest positive value:

- 符号位 $S = 0$ (正数)
- 指数位 E 尽可能大: E 不能全为1 (全为1表示无穷大) , 所以最大为 11111110 (254) 。
- 尾数位 M 尽可能大: 23位全部为 1, $M=1.11...1$
- $N_{\max} = 1.11...1 \times 2^{254-127} = (2-2^{-23}) \times 2^{127} \approx 3.4028235 \times 10^{38}$



IEEE 754 float (4 Bytes)



– Precision:

- float 使用 23 位来存储尾数，加上隐含的 “前导1”，实际有 24 位的有效二进制精度。
- 计算能表示的十进制精度可以通过公式 $\log_{10}(2^n)$ 来估算，其中 n 是二进制位数。
 $\log_{10}(2^{24}) \approx \log_{10}(16,777,216) \approx 7.225$
- 这意味着 24 位二进制精度大约相当于 7.225 位十进制精度。因此，我们保守地说，精度是 7 位十进制数字。
- 这表示你对一个 float 变量赋值时，无论你输入的数字有多长，计算机最多只能安全地存储大约前7位有效数字。第8位有效数字开始 “失真” 。



IEEE 754 float (4 Bytes)

例 1：存储一个7位数字

```
float f = 123.4567; // 7位有效数字
```

这个值很可能被精确存储，没有误差。

例 2：存储一个超过7位的数字

```
float f = 123.456789; // 9位有效数字
```

```
printf("%.9f\n", f); // 输出可能类似：123.456787109
```

你希望存储 123.456789，但 float 的精度只有 ~7 位。计算机会将这个数字舍入到它能表示的最近似的值。当你打印出来时，你可能会看到

123.456787109。注意：前7位数字 (123.4567) 很可能是正确的。从第8位开始 (89 vs 87109)，后面的数字就失真了，它们是舍入误差的结果，不再可靠。



IEEE 754 float (4 Bytes)

例 3：大数的问题（更明显）

```
float f = 123456789.0f; // 9位有效数字 (123456789)
```

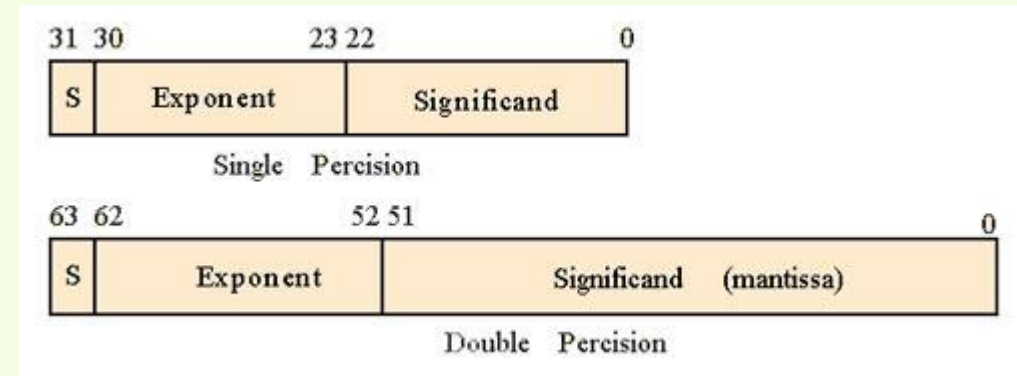
```
printf("%f\n", f); // 输出很可能为：123456792.000000
```

你希望存储 123,456,789。但 float 的精度只有7位有效数字，意味着它无法精确区分千万级别数字的个位和十位。它只能将这个大数舍入到最接近的、它能表示的值，结果是 123,456,792。你会发现，前7位数字 1234567 是稳定的，但后面的 89 变成了 92，已经不再准确。



Float/double precision

- Float and double are not precise
 - The storage is not accurate.
 - Floating operation uses specific hardware
- Use double if no specific requirement





Exe

- What's the height of LBJ
- How 6 feet 9 inches 2.06m?
- **Meter = (foot +inch/12)*0.3048**
- Write a program

```
int main()
{
    printf("input foot and inch:\n");
    int foot;
    int inch;
    ...
    return 0;
}
```



Los Angeles Lakers | #6 | Forward

LeBron James

PPG	RPG	APG	PIE	HEIGHT	WEIGHT
25.0	7.7	7.8	19.1	6'9" (2.06m)	250lb (113kg)
				AGE	BIRTHDATE
				36 years	December 30, 1984



Character Types

- Char is also an integer type

- 'a', '1'

- %c in scanf and printf

- ASCII code

- '1'的ASCII编码是49,
 - 当变量a==49, a的值就是'1'

```
char ch;  
int i;  
i = 'a';           // i is now 'a', its value is 97  
ch = 65;           // ch is now 'A' , its value is 65  
ch = ch + 1;       // ch is now 'B', its value is 66  
ch++;              //ch is now 'C', its value is 67
```

```
for(ch = 'A'; ch <= 'Z'; ch++)  
{  
    printf("%c\n",ch);  
}
```



Boolean

- Boolean: {true, false}
- There is no Boolean data type in C
- Any integer $\neq 0$ is considered true
- 0 is considered false

```
int a= 3;
```

```
int b = !a; // b=0
```



Constants int

- Integer constants: (整型常量)

- Decimal: 11,12345

- Declare a named constant:

- Using **const** keyword

```
const int PAIR=2
```

- Using **#define** pre-processor directive

```
#define PAIR 2
```

预处理指令，在编译之前，把所有PAIR简单替换成 2。

```
int main()
{
    int n;
    const int TIMES = 2;
    scanf("%d",&n);
    printf("Output is %d\n",TIMES*n);
}
```




Constants

- Character constants
- String constants
 - "I am a string"
 - Character array
 - always null ('\0') terminated.
 - (see <string.h> for string functions)